# LAPORAN JOBSHEET
# “4-Link Navigation”
# MATA KULIAH PEMROGRAMAN BERBASIS FRAMEWORK

M. FATIH AL GHIFARY

2341720194 / TI-3A

**PROGRAM STUDI D-IV TEKNIK INFORMATIKA**

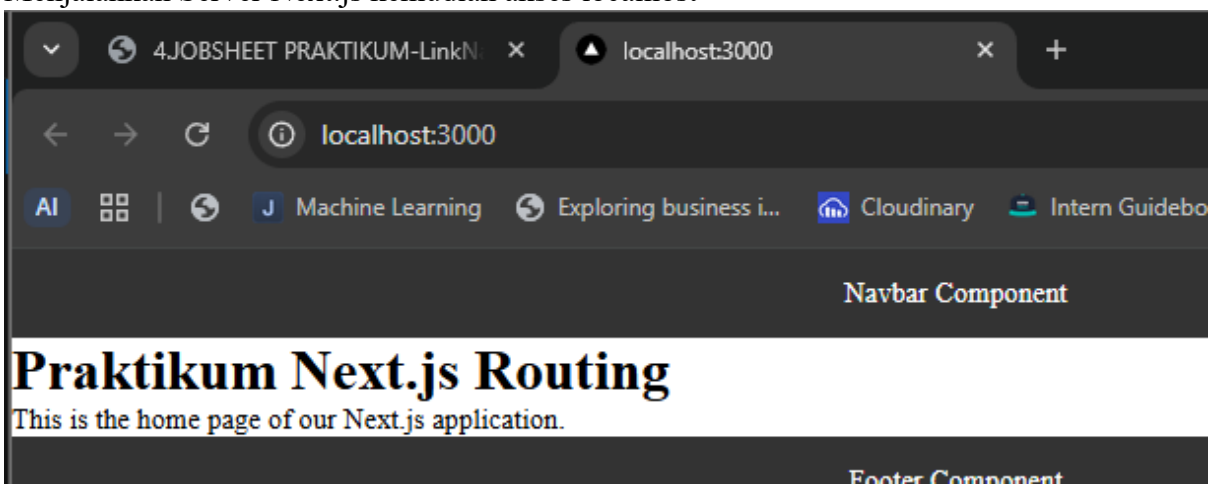
**JURUSAN TEKNOLOGI INFORMASI**

**POLITEKNIK NEGERI MALANG**

**2026**

## Langkah 1 – Menjalankan Project

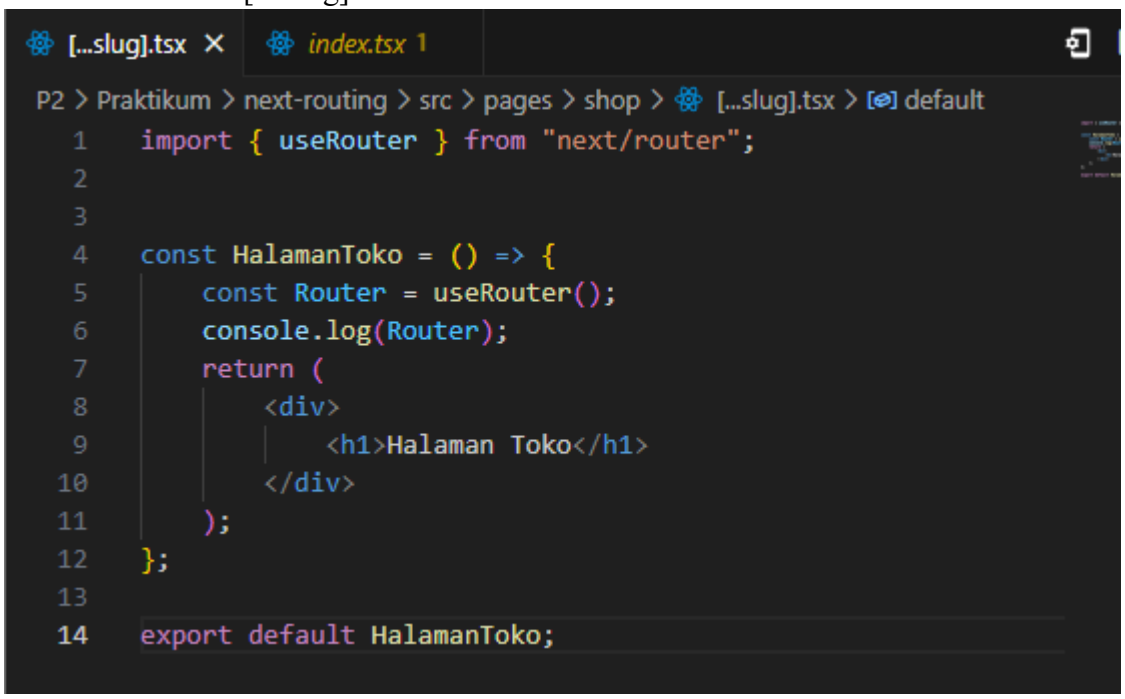
1. Menjalankan Server Next.js kemudian akses localhost




## Langkah 2 – Membuat Catch-All Route

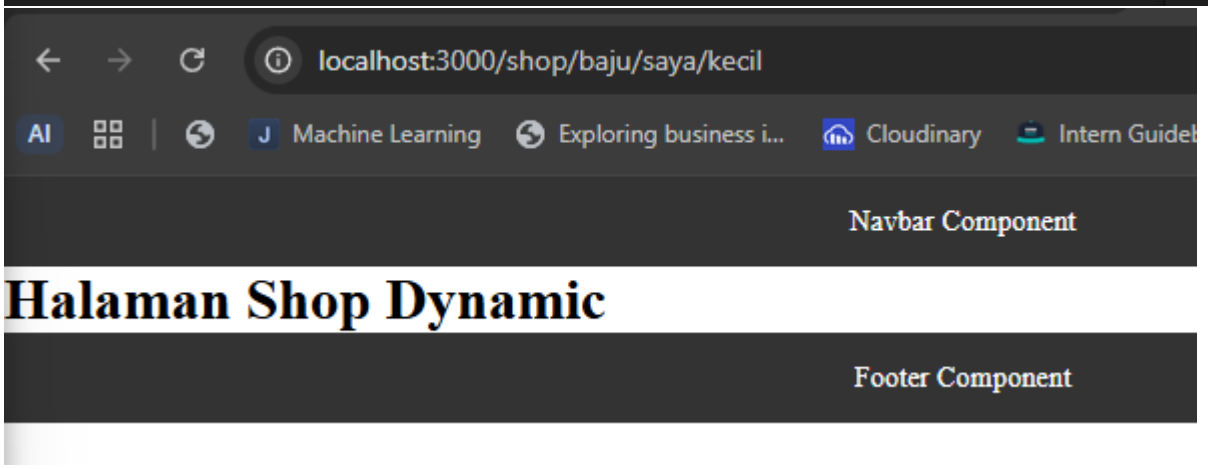
1. Buat folder shop dan file […slug].tsx:

2. Modifikasi Isi file […slug].tsx




3. Cek menggunakan console.log apakah nilai segment berhasil didapat

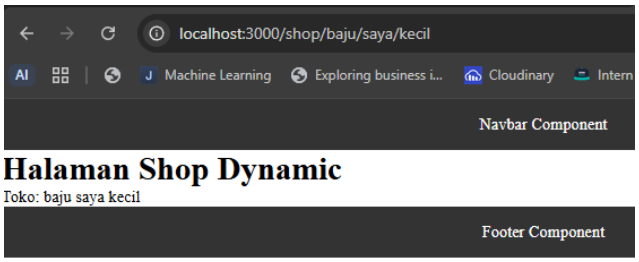
```
[id].tsx U    [...slug].tsx U ×
P2 > Praktikum > next-routing > src > pages > shop > [...slug].tsx > HalamanShop
1   import { useRouter } from "next/router";
2
3   const HalamanShop = () => {
4     const router = useRouter();
5     console.log(router.query);  ×14 { slug: [ 'baju', 'saya', 'kecil' ] } { slug: [ 'baju
6
7     return <h1>Halaman Shop Dynamic</h1>;
8   };
9
10  export default HalamanShop;
```




4. Modifikasi [...slug].tsx untuk menampilkan nilai query

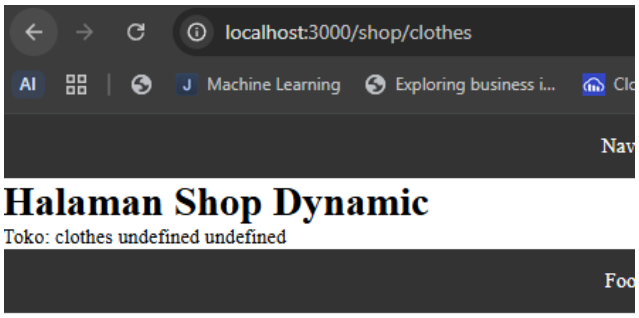
```
[id].tsx U    [...slug].tsx U ×
P2 > Praktikum > next-routing > src > pages > shop > [...slug].tsx > HalamanShop
1   import { useRouter } from "next/router";
2
3   const HalamanShop = () => {
4     // const router = useRouter();
5     // console.log(router.query);
6     const { query } = useRouter();
7
8     return (
9       <div>
10        <h1>Halaman Shop Dynamic</h1>
11        <p>Toko: {`${query.slug && query.slug[0]+" "+query.slug[1]}`}</p>
12      </div>
13    );
14  };
15
16  export default HalamanShop;
```

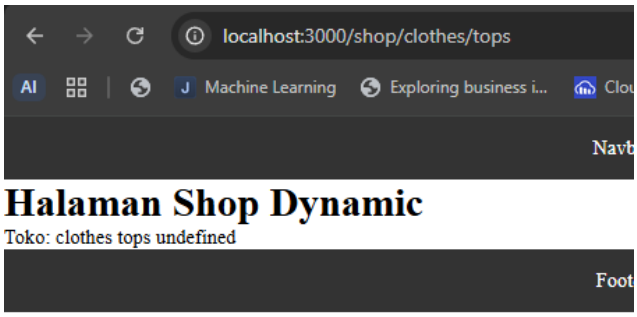
5. Cek hasil di browser

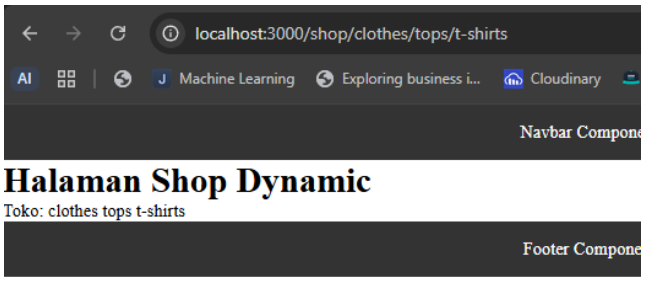



## Langkah 3 – Pengujian Catch-All Route

1. Coba coba di browser



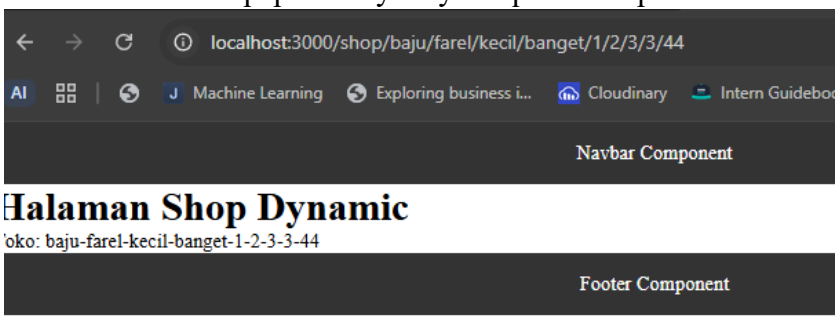




2. Jika dilihat ada yang terbaca undifined dan ada yang tidak terbaca ini dikarena segmennya dibatasi Cuma array[0] dan array[1]. Solusinya bagaimana ? Modifikasi […slug].tsx menjadi berikut

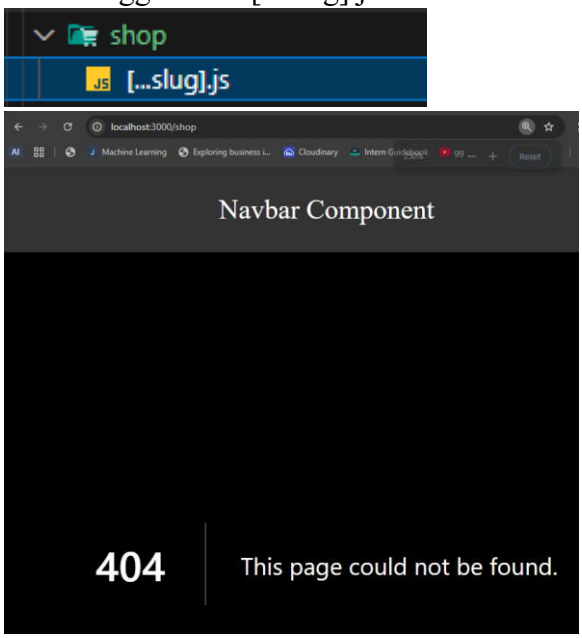
```
return (
  <div>
    <h1>Halaman Shop Dynamic</h1>
    {/* <p>Toko: {`${query.slug && query.slug[0]+" "+query.slug[1]+" "+query.slug[2]}`}</p>
    <p>Toko: {Array.isArray(query.slug) ? query.slug.join(" ") : query.slug}</p>
  </div>
);
```

3. Cek browser: Berapapun banyaknya seqment tetap terbaca

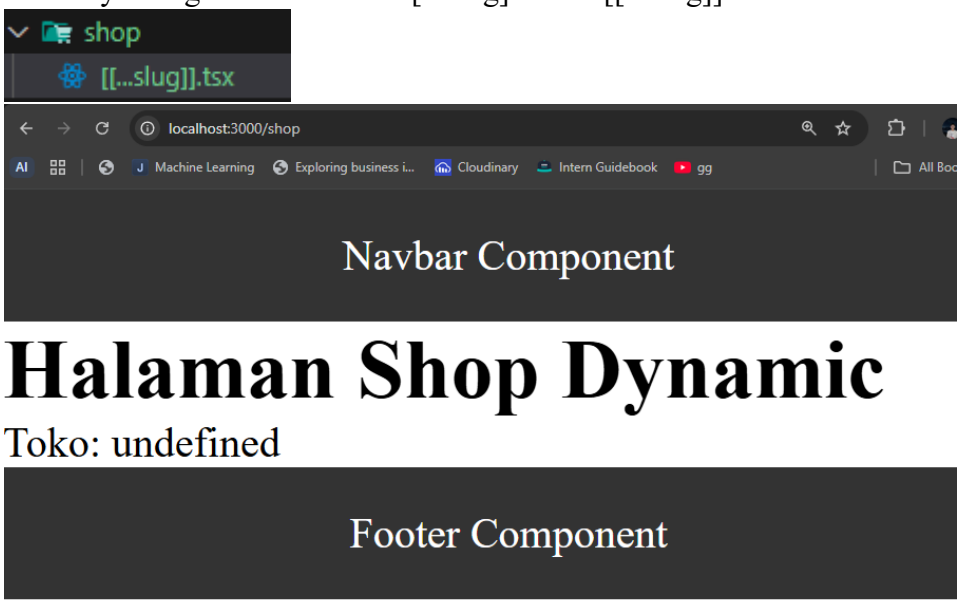



## Langkah 4 – Optional Catch-All Route

1. Jika menggunakan [...slug].js maka ketika mengakses shop akan terjadi error




2. Solusinya dengan Rename file: [...slug].tsx → [[...slug]].tsx



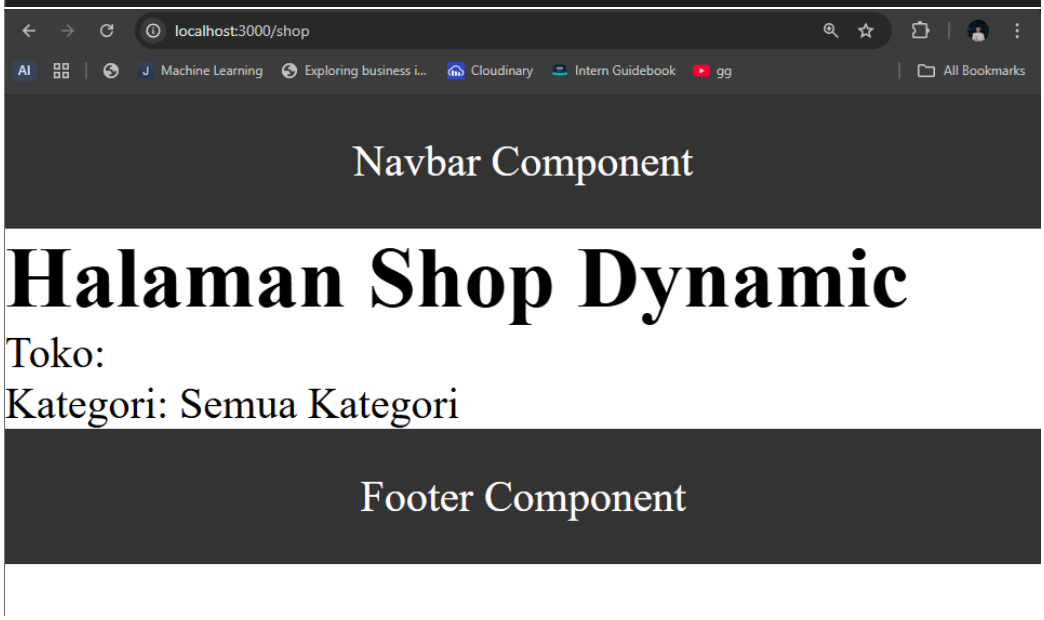
## Langkah 5 – Validasi Parameter

1. Tambahkan validasi agar tidak error saat slug kosong:

[id].tsx U  [[...slug]].tsx U

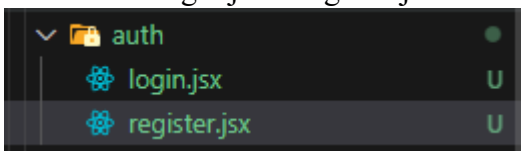
P2 > Praktikum > next-routing > src > pages > shop > [[...slug]].tsx > HalamanShop

```
import { useRouter } from "next/router";

const HalamanShop = () => {
  // const router = useRouter();
  // console.log(router.query);
  const { query } = useRouter();

  return (
    <div>
      <h1>Halaman Shop Dynamic</h1>
      {/* <p>Toko: {`${query.slug && query.slug[0]+" "+query.slug[1]}`}</p> */}
      <p>Toko: {Array.isArray(query.slug) ? query.slug.join("-") : query.slug}</p>
      <p>
      Kategori: {query.slug ? query.slug[0] : "Semua Kategori"}
      </p>
    </div>
  );
};

export default HalamanShop;
```




## Langkah 6 – Membuat Halaman Login & Register

1. Buat file: o login.jsx o register.jsx



2. Modifikasi login.tsx

```
[[...slug]].tsx U   login.tsx U ×   register.tsx U   Extension: GitHub Copilot Cha
P2 > Praktikum > next-routing > src > pages > auth > login.tsx > default
1   import Link from "next/link";
2
3   const HalamanLogin = () => {
4
5       return (
6           <div>
7               <h1>Halaman Login</h1>
8               <Link href="/auth/register">Ke Halaman Login</Link>
9           </div>
10
11      );
12  };
13
14  export default HalamanLogin;
```

3. Modifikasi register.tsx

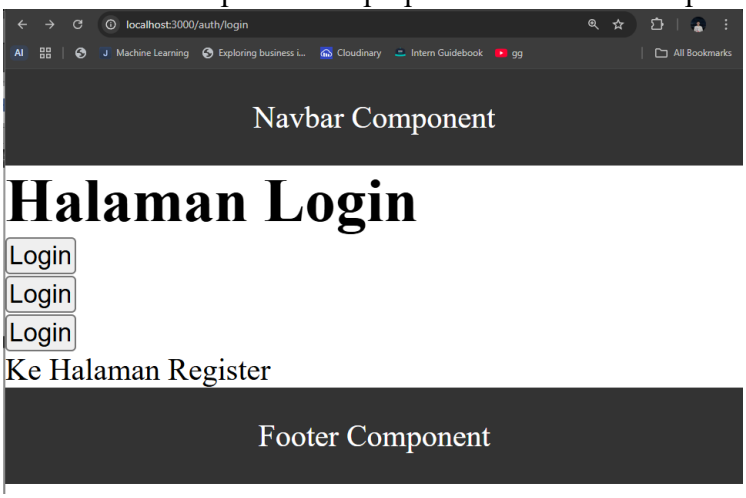
```
[[...slug]].tsx U   login.tsx U   register.tsx U ×   Extension: Gi
P2 > Praktikum > next-routing > src > pages > auth > register.tsx > default
1   import Link from "next/link";
2
3   const HalamanRegister = () => {
4     return (
5       <div>
6         <h1>Halaman Register</h1>
7         <Link href="/auth/login">Ke Halaman Login</Link>
8       </div>
9     );
10  };
11
12  export default HalamanRegister;
```

## Langkah 7 – Navigasi Imperatif (router.push)

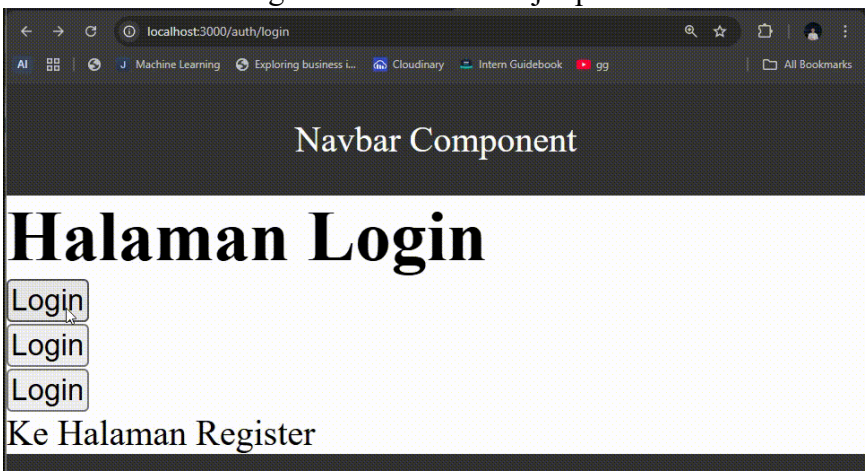
1. Tambahkan button login:

```
[[...slug]].tsx U   login.tsx U ×   register.tsx U   Extension: GitHub Copilot Chat
P2 > Praktikum > next-routing > src > pages > auth > login.tsx > HalamanLogin > handlerLogin
1   import Link from "next/link";
2   import { useRouter } from "next/router";
3
4   const HalamanLogin = () => {
5       const {push} = useRouter();
6       console.log(push);
7
8       const handlerLogin = () => {
9           push("/produk");
10      }
11      return (
12    <div>
13      <h1>Halaman Login</h1>
14      <button onClick={handlerLogin}>Login</button> <br />
15      <button onClick={() => push('/produk')}>Login</button> <br />
16      <button onClick={() => handlerLogin()}>Login</button> <br />
17      <Link href="/auth/register">Ke Halaman Register</Link>
18    </div>
19  );
20  };
21
22  export default HalamanLogin;
```

2. Klik tombol dan perhatikan perpindahan halaman tanpa reload.




3. Jika di klik button login maka akan menuju /produk

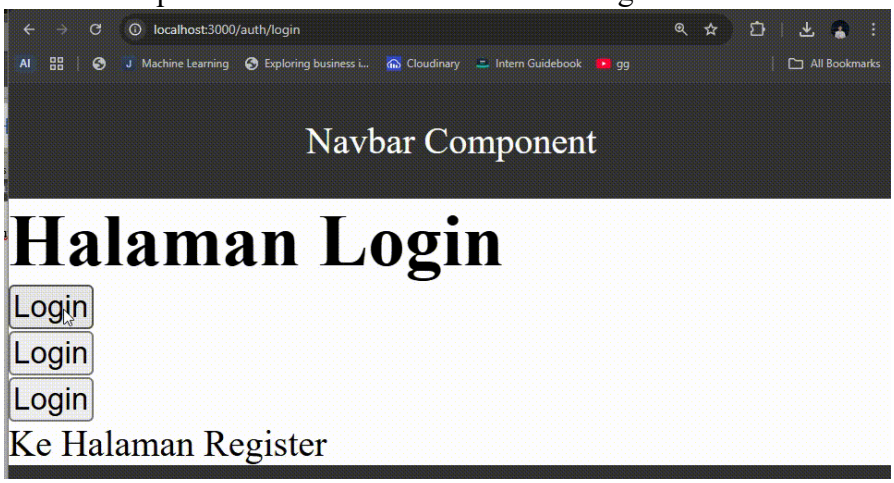



## Langkah 8 – Simulasi Redirect (Belum Login)

1. Di halaman product, pada index.tsx tambahkan beberapa code berikut:

```
[[...slug]].tsx U   login.tsx U   index.tsx 2, U   register.tsx U
P2 > Praktikum > next-routing > src > pages > produk > index.tsx > ProductPage
1   import { useRouter } from "next/router";
2   import { useEffect, useState } from "react";
3
4   const ProductPage = () => {
5       const [islogin, setIsLogin] = useState(false);
6       const {push} = useRouter();
7
8       useEffect(() => {
9           if(!islogin) {
10              push("/auth/login");
11          }
12      },[]);
13      return (
14          <div>
15              <h1>Halaman Produk</h1>
16          </div>
17      );
18  };
19
20  export default ProductPage;
```

2. Jika Akses /product → otomatis diarahkan ke login.

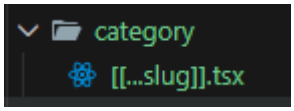



Autodirect ke login jika belum login

## TUGAS PRAKTIKUM

### Tugas 1

1. **Buat catch-all route**




- Isi dari [[…slug]].tsx

2. **Tampilkan seluruh parameter URL dalam bentuk list.**

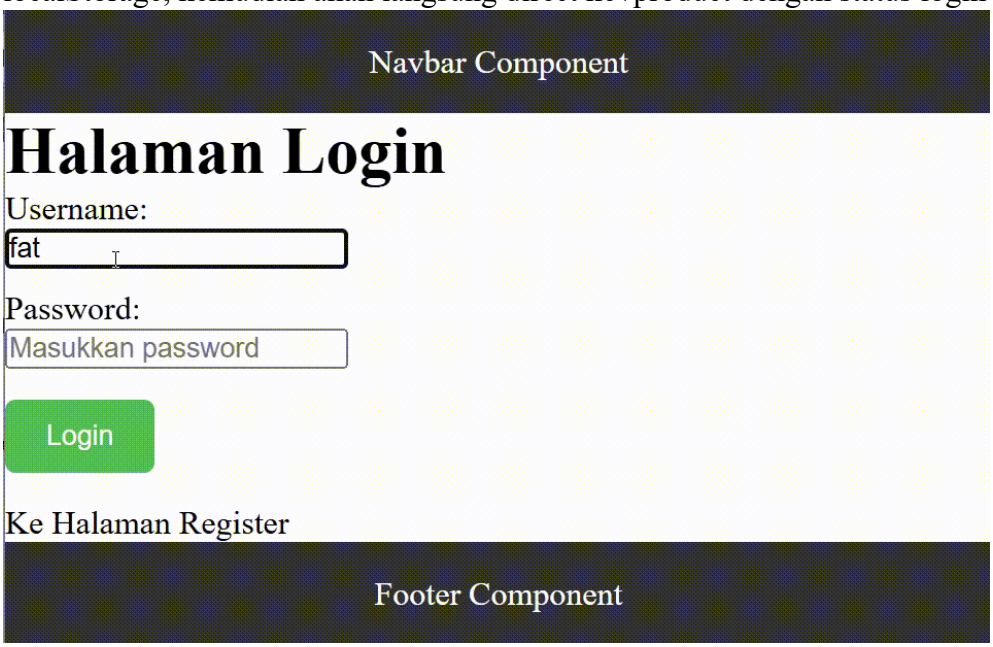
**Tugas 2**

1. **Buat navigasi:**
   - **Login → Product (imperatif)**

     Saya menggunakan logika sederhana untuk login menggunakan localStorage

```
const HalamanLogin = () => {
  const { push } = useRouter();
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");

  const handlerLogin = () => {
    // Validasi login sederhana
    if (username === "fatih" && password === "123") {
      localStorage.setItem("isLogin", "true");
      push("/product"); // Navigasi imperatif ke halaman produk
    } else {
      setError("Username atau password salah!");
    }
  };
```

     Jadi ketika user login maka username dan password akan disimpan di localStorage, kemudian akan langsung direct ke /product dengan status login true.




   - **Login ↔ Register (Link)**

Code di dalam login.tsx

```
<div style={{ marginTop: "15px" }}>
  <Link href="/auth/register">Ke Halaman Register</Link>
</div>
</div>
```

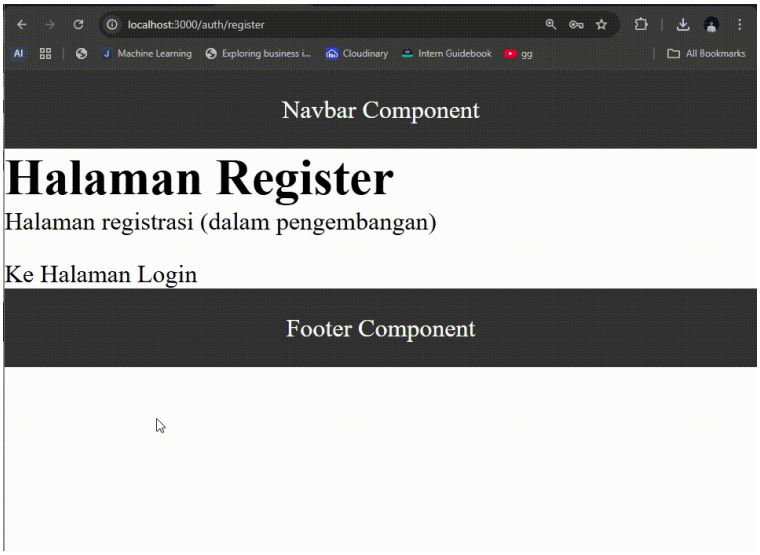
Code di dalam register.tsx

login.tsx 5, U | index.tsx 1, U | register.tsx 1, U

P2 > Praktikum > next-routing > src > pages > auth > register.tsx > ...

```
1  import Link from "next/link";
2
3  const HalamanRegister = () => {
4    return (
5      <div>
6        <h1>Halaman Register</h1>
7        <p>Halaman registrasi (dalam pengembangan)</p>
8
9        <div style={{ marginTop: "15px" }}>
10         <Link href="/auth/login">Ke Halaman Login</Link>
11       </div>
12     </div>
13   );
14 };
15
16 export default HalamanRegister;
```

Ketika di browser



**Tugas 3**

1. **Terapkan redirect otomatis ke login jika user belum login.**
   Code di dalam index.tsx dari /product

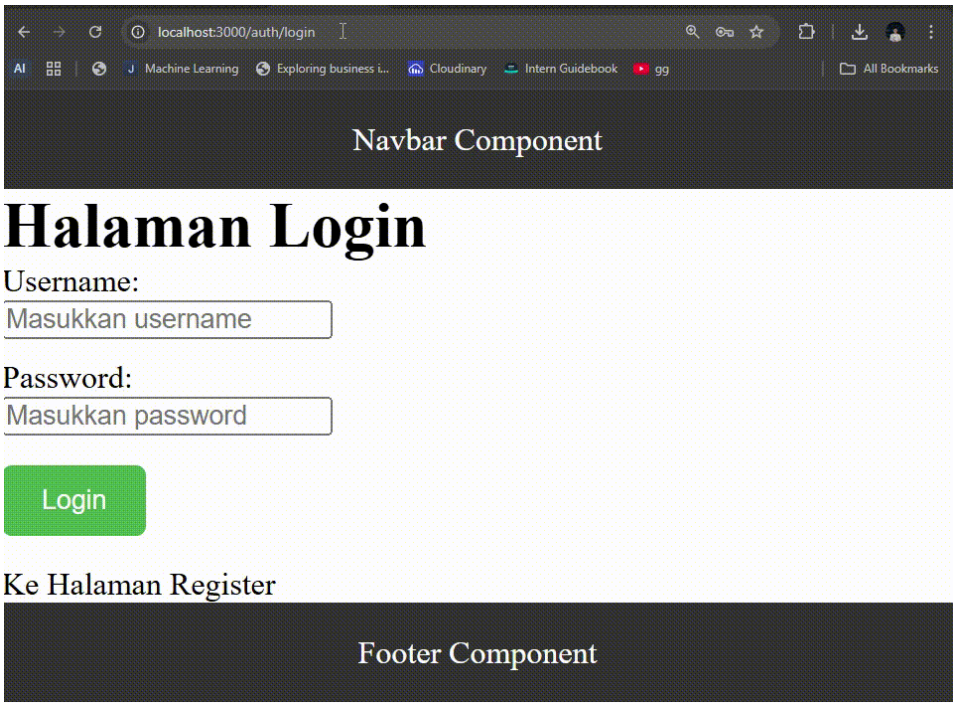
```
login.tsx 5, U    index.tsx 1, U ×    register.tsx 1, U
P2 > Praktikum > next-routing > src > pages > product > index.tsx > ProductPage
import { useRouter } from "next/router";
import { useEffect, useState } from "react";

const ProductPage = () => {
    const { push } = useRouter();
    const [isLoading, setIsLoading] = useState(true);

    useEffect(() => {
        const loginStatus = localStorage.getItem("isLogin");
        if (loginStatus !== "true") {
            push("/auth/login");
        } else {
            // eslint-disable-next-line
            setIsLoading(false);
        }
    }, [push]);

    const handleLogout = () => {
        localStorage.removeItem("isLogin");
        push("/auth/login");
    };

    if (isLoading) {
        return <p>Loading...</p>;
    }
```

- Saya menggunakan logika sederhana juga menggunakan useEffect dari react. Jadi, ketika masuk ke /product akan dicek terlebih dahulu loginStatus nya (diambil dari isLogin yg disimpan dari halaman login sebelumnya, yaitu di localStorage). Kemudian jika true maka akan masuk halaman /product, namun jika loginStatus tidak sama dengan true maka akan redirect otomatis ke halaman login.
- Saya juga menggunakan state setIsLoading untuk menghilangkan ESlint, karena kata AI ESlint yg saya alami itu karena "setState di dalam useEffect karena dianggap tidak optimal (meski kode tetap berjalan)." Memang tadi sempet berjalan ketika saya menggunakan parameter isLogin = true/isLogin = false untuk logika redirectnya, namun terdapat ESlint yang mengganggu, sehingga saya menggunakan

```
if (isLoading) {
    return <p>Loading...</p>;
}
```

  untuk mengatasinya.

2. Cek di Browser




## Pertanyaan Evaluasi

1. Apa perbedaan [id].js dan [...slug].js?
   [id].js digunakan untuk dynamic route dengan satu parameter saja, misalnya /produk/1 atau /user/10. Sedangkan [...slug].js digunakan untuk catch-all route, yaitu menangkap banyak segmen URL sekaligus, misalnya /blog/react/nextjs/tutorial.

2. Mengapa slug berbentuk array?
   Karena pada catch-all route ([...slug].js), URL bisa memiliki lebih dari satu segmen. Next.js menyimpannya dalam bentuk array, misalnya URL /blog/react/nextjs akan menjadi ['blog','react','nextjs'].

3. Kapan sebaiknya menggunakan Link dan router.push()?
   Link digunakan untuk navigasi biasa melalui klik link di UI. Sedangkan router.push() digunakan untuk navigasi melalui kode (programmatically), misalnya setelah login berhasil atau setelah submit form.

4. Mengapa navigasi Next.js tidak me-refresh halaman?
   Karena Next.js menggunakan client-side navigation dengan React. Saat berpindah halaman, Next.js hanya mengganti komponen yang diperlukan tanpa reload seluruh halaman, sehingga navigasi terasa lebih cepat.